

Arquitetura de Computadores I

RISC-V e codificação

REALIZAR APÓS posição 37: Codificação de Instruções

Posições na playlist 32 a 37 | 120 minutos | 100 pontos

Curso principal: MC404 - 2021s1 - Organização Básica de Computadores e Linguagem de Montagem | <https://www.youtube.com/playlist?list=PLEUHFTHcrjmswfeq7QEHskgkT6HER3gK6>

Conteúdo verificável: RISC-V; instruções básicas; memória; saltos; simulação; codificação

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

RISC-V: execução

Considere RV32I: `addi x5, x0, 7`; `addi x6, x0, 3`; `add x7, x5, x6`; `sub x8, x5, x6`. (a) Dê registradores finais. (b) Explique `x0`. (c) Mostre como verificar em simulador passo a passo.

Desenvolvimento / justificativa / esquema:

Memória e saltos

Escreva pseudocódigo assembly RV32I para somar quatro palavras de 32 bits começando no endereço guardado emx10, resultado emx11. (a) Use lw, addi, add e ramo. (b) Dê deslocamentos/avanço. (c) Explique por que aumentar ponteiro em 1 seria incorreto para palavras consecutivas.

Desenvolvimento / justificativa / esquema:

Codificação

Formato: `imm[11:0]` em 31...20, `rs1` em 19...15, `funct3` em 14...12, `rd` em 11...7, `opcode` em 6...0. Para `addi`, `funct3`=000, `opcode`=0010011. (a) Codifique `addi x5, x0, -1`. (b) Dê hexadecimal. (c) Explique extensão de sinal e limite do imediato 12 bits.

Desenvolvimento / justificativa / esquema:

Laço e controle

Traduza para assembly RV32I: enquanto $x5 < x6$, faça $x5 = x5 + 2$; ao final $x7 = x5$. (a) Use comparação assinada e saltos. (b) Trace $x5 = 1$, $x6 = 6$. (c) Compare com interpretação sem sinal quando $x5 = -1$.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - RISC-V: execução (15 pontos)

$x5=7$, $x6=3$, $x7=10$, $x8=4$; $x0$ permanece 0. Tentativa de gravar em $x0$ é descartada pelo modelo arquitetural. Simulação: inspecionar PC, instrução e valores após cada passo; testar operando negativo e overflow dentro da largura se solicitado. A instrução `addi` usa imediato; não lê memória pelo valor 7.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Memória e saltos (25 pontos)

Zerar $x11$; contador 4; laço `lw t0, 0(x10); add x11, x11, t0; addi x10, x10, 4; decrementar contador; ramo enquanto diferente de 0`. Cada palavra ocupa 4 bytes, logo incremento 4. Fornecer nomes de registradores temporários e rótulos equivalentes. A validade de alinhamento e área acessível faz parte do contrato; somar endereços não soma seus conteúdos.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Codificação (25 pontos)

$imm=111111111111$, $rs1=00000$, $funct3=000$, $rd=00101$, $opcode=0010011$. Palavra `0xFFFF00293`. O imediato representa -1 em 12 bits e é estendido pelo sinal para 32 bits. Faixa assinada $-2048 \dots 2047$. Não codificar -1 como sinal-magnitude nem usar um campo 32 bits dentro dessa instrução.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Laço e controle (35 pontos)

`LOOP: bge x5, x6, END; addi x5, x5, 2; jal x0, LOOP; END: add x7, x5, x0`. Trace $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$, resultado $x7=7$. Com -1 , comparação assinada considera menor que 6; sem sinal, `0xFFFFFFFF` é grande e a decisão difere. Explicitar variante do ramo evita resultado ambíguo. O salto com destino $x0$ descarta endereço de retorno.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.